

압축 성능 평가 지표

결론 요약

1. 압축 성능의 단일 표준 지표는 존재하지 않는다. 학계·업계의 사실상 표준은 2축 동시 보고: ①압축률(배수/%) + ②품질 보존율(%). "N배 압축에 원래 성능 M% 유지" 형태가 지배적. ([Prompt Compression in the Wild](#)는 압축률/다운스트림 성능/grounding/정보 보존 4축을 명시.) 여기에 ③효율성(지연시간·메모리·비용) 축을 함께 보고하는 관행이 최근 추세 — 상세는 §3.1.
 2. BranchPort의 압축은 두 종류이고 평가 지표를 분리해야 한다:
 - 구조적 압축(레코드 ~1,000개 → 턴 10~80개, 손실 없는 접기) → 그래프 요약·코드 폴딩 분야 지표
 - 내용 압축(LLM 요약 생성) → 요약 품질·사실성 지표
 3. 정답(참조) 요약이 없는 우리 상황에서 로컬 실행 가능한 최적 후보는 SummaC(NLI 기반 사실적 일관성, `pip install summac`, CPU 가능).
 4. Claude Code /`compact` 자체를 정량 평가한 공식 벤치마크는 없다 — 커뮤니티 정성 보고 수준. 우리가 압축 품질을 정량 측정하면 그 자체가 차별점.
 5. 문제 설정이 거의 동일한 선행연구 2건을 확보: Autofolding/TASSAL(코드 폴딩=손실 없는 접기), Githru(git DAG 집약 시각화 + 사용자 연구). 결과보고서 [related work](#)에 직접 인용 권장.
 6. CLI 코딩 에이전트의 컨텍스트 압축을 직접 다룬 검증된 선행연구를 추가 확보: OPENDEV ACC(5단계 점진 압축, 관측값 peak 54% 절감, "아카이브로 사실상 무손실" — 우리와 동일 철학), SelfCompact(모델이 압축 시점 자체 판단, 비용 30~70% 절감 + 정확도 상승), OpenHands condenser 실측(SWE-bench 성공률 유지 + 비용 절반), Control Under Compression(예산 75%까지 안전, 50~35%에서 급락하는 신뢰도 경계). 최근 보고 관행은 단일 수치 대신 트레이드오프 곡선/frontier(§3.5). 상세는 §3.3~3.5.
-

1. 구조적 압축(접기)의 지표 — 그래프 요약·코드 폴딩·HCI

지표	정의/측정	BranchPort 적용	출처
압축률	$1 - (\text{턴 수} / \text{원시 레코드 수})$ 또는 배수	즉시 계산 가능. 실측 기준 중앙값 161→10, 최대 1,007→80 (~94% 감소)	Graph Summarization Survey
재구성 완전성	요약본에서 원본 복원 시 오차 (reconstruction error)	접기는 무손실이므로 "재구성 오차 0 — 턴을 펼치면 원본 레코드 100% 복원" 문구로 사용	CGS
질의 보존	요약본에서 원본과 동일 질의에 답 가능한지	"턴 뷰만으로 '어떤 파일이 수정됐나' 질의에 100% 답변" 자동 검증 가능	CGS
화면/스크롤 감소율	화면당 표시 항목 수, 스크롤 길이 비교	원시 JSONL 뷰 vs 턴 뷰의 스크롤 횟수 비교 → "스크롤 N% 감소"	Event Sequence 시각화 서버
과제 완료 시간 A/B	동일 질의("실패한 툴킷 찾기")를 원시 로그 vs 도구로 수행, 시간·정답률 비교	Githru가 개발자 12명 통제 실험으로 이 방법 사용. 팀 내부 소규모로 재현 가능 — 심사위원 설득력 최고 후보	Githru (IEEE VIS 2020)
PREVis 가독성 설문	4차원 11문항 검증된 설문 척도	압축 전/후 뷰에 팀원 설문 → 정량 가독성 점수	PREVis

선행연구 직접 인용 포인트:

- Autofolding/TASSAL — "폴딩 후 길이가 원본의 특정 비율 이하 + 안 접힌 부분이 가장 중요한 내용"으로 압축 문제를 정의. "코드 폴딩의 아이디어를 대화 로그에 적용했다"는 서사에 사용. ([논문](#))
- Githru — git 커밋 DAG를 집약(Context-Preserving Squash Merge)해 시각화 + 과제 완료 시간으로 평가. 문제 설정·평가 방법 모두 우리와 동일 구조. ([논문](#))

2. 내용 압축(LLM 요약)의 지표 — 요약 품질·사실성

참조(정답 요약) 없이 쓸 수 있는 것 — 우리 상황의 주력

지표	방식	로컬 실행	권고
SummaC	NLI로 원문×요약 문장 합의 행렬 → 사실적 일관성	✅ pip install summac, CPU 가능	1순위. "요약이 원문 로그에서 합의되는 내용만 담는가"
QAFactEval	QA 기반 사실성 (성능은 최고 수준)	⚠️ torch 1.6 종속, 최신 환경 비호환	비추천 (유지보수 부담)
QuestEval	QG+QA 파이프라인	⚠️ 모델 2개, 무거움	보류
G-Eval (LLM-as-judge)	LLM에 CoT 채점 지시	△ 요약 생성에 쓰는 LLM 재활용	보조. 신뢰성 논쟁 있음 — 쓴다면세밀한 점수 대신 이진 판정 (근거)

대화 로그 특화 지표 (일반 요약과 다른 부분)

- 화자/주체 귀속 오류율: 대화 요약 모델 출력의 33~42%가 "누가 무엇을 했는지" 혼동 오류 포함 (연구). 우리 버전: 토크 주체·대상 파일 오귀속 비율 — 세션 로그 요약에서 특히 중요.
- Key-fact recall: 수정된 파일명·실행 명령 같은 핵심 사실이 요약에 남아있는 비율 (정보 보존 논문). 우리 데이터는 토크 레코드에서 정답을 기계적으로 추출할 수 있어 자동 채점 가능 — ROUGE보다 훨씬 우리에게 유리한 지표.
- ROUGE류는 뉴스 기사용 설계라 대화 요약에 부적합하다는 것이 정설(SAMSum 저자 지적) — ROUGE 단독 주장은 학계 기준 약한 근거. 골드셋(사람이 쓴 요약)을 소량 만들면 ROUGE-L+BERTScore 보조 사용 가능.
- 평가 차원 프레임: SummEval 4차원(coherence/consistency/fluency/relevance)을 채점 체크리스트로 인용 (SummEval, DialSummEval).

3. 컨텍스트 압축 연구의 지표·보고 관행 (비교 문맥용)

3.1 효율성(Efficiency) 지표 — 우리가 쓸 수 있는 것만

컨텍스트 압축 논문들이 품질 측과 함께 보고하는 효율성 측 중 BranchPort에 해당하는 것:

지표	정의	BranchPort 적용	출처
토큰 절감률	압축 후/전 토큰 수 비율 (예: 54% 절감, 4× 압축)	✅ LLM 요약 기능의 입력 토큰 절감으로 보고 가능	ACON · OPENDEV
비용 절감	질문(태스크)당 토큰 비용(USD)	✅ 요약 기능 도입 시 산출 가능	SelfCompact (30~70% 절감 보고)
압축 시간	압축(요약 생성) 자체에 걸리는 시간 — 순절감 계산의 차감 항목	△ 요약 생성 시간만 해당	Prompt Compression in the Wild · 경량 압축(엠티)

그 외 효율성 지표(GPU 메모리·KV 캐시·처리량·TTFT/TBT·percentile 지연시간)는 모델 서빙 지표라 로컬 시각화 도구인 우리와 무관 — §4.3의 제외 목록 참고. 용어 주의: 일부 2차 자료가 쓰는 "FTIT"는 비표준 표기이며 표준 용어는 TTFT.

3.2 성능(Effectiveness) 지표 — 압축 연구 관행

- F1 / Exact Match(EM): RAG·QA형 평가의 표준. 압축 전후 점수 비교로 "품질 보존율"을 산출 — §2의 key-fact recall이 우리 버전에 해당.
- BLEU / ROUGE-L: 압축 표현에서 원문을 복원(reconstruction)하는 품질 측정에 사용 (PCC). 우리의 구조적 압축은 무손실이라 재구성 오차 0으로 같음(\$1).
- 작업별 점수: 수학·에이전트 탐색 등 다운스트림 태스크 점수 변화로 압축 품질을 간접 측정 (SelfCompact 방식).

3.3 대표 연구 수치 (원논문 검증 완료)

- SelfCompact (arXiv 2606.23525, 2026-06 · 코드): 모델 스스로 압축 시점을 판단(발동/억제 루브릭 + 요약 토크). 무요약 대비 수학 최대 +18.1점, 에이전트 탐색 +5~9점, 질문당 비용 30~70% 절감. 50~100k 토큰 궤적을 1~3k로 요약(20~80× 축소). → "언제 압축하느냐"가 품질을 좌우한다는 근거로 인용 가치.

- PCC(Pretraining Context Compressor, [ACL 2025](#), Microsoft): 임베딩 기반(소프트 프롬프트) 압축. 4× 압축 시 평균 F1 63.62 vs 원본 컨텍스트 66.29 (≈96% 유지, Llama-3-8B 기준), 4×/16×를 권장 압축률로 제시. 극단 압축(256×)은 정보 손실 급증 — LLMingua의 경향과 일치.
- LLMingua 계열: 압축률(2x~20x) + 다운스트림 성능 유지율 + 지연시간을 함께 보고. 20x 이하 추출식 압축은 95%+ 정확도 유지, 극단 압축(480x)은 62~73%로 급락. ([LLMingua-2](#))
- ACON(Microsoft, 2025): 에이전트 컨텍스트 압축을 peak 토큰 26~54% 감소 + 태스크 성공률 유지/향상으로 평가. ([논문](#))
- 대화 메모리 벤치마크: LoCoMo, LongMemEval, MemBench — QA 정확도 기반. 우리가 그대로 쓰긴 어렵지만 "압축률↑ ↔ 수치·시간 정보 충실도↓" 트레이드오프 보고 관행 참고.
- needle-in-haystack 변형: "요약 과정에서 소실된 정보가 나중에 필요해지는" 시나리오 평가 ([ARC](#)).

3.4 CLI 코딩 에이전트 특화 지표 — 우리와 가장 가까운 도메인

- OPENDEV의 Adaptive Context Compaction(ACC) ([arXiv 2603.05344](#) §2.3.6): 톨 관측값이 컨텍스트의 70~80%를 차지한다는 전제에서, 5단계 점진 압축(임계값 70/80/85/90/99%)으로 관측값을 active→faded→archived로 전이. 오래된 톨 결과를 ~15토큰짜리 참조 포인터로 치환. 정량 결과: 관측값 peak 컨텍스트 소비 약 54% 절감, 통상 30턴 세션에서 긴급 압축(emergency compaction)을 대부분 제거 (원문 확인 완료).
- BranchPort 서사와의 접점: OPENDEV는 전체 히스토리를 아카이브 파일로 남기고 요약에 경로를 주입해 "사실상 무손실(effectively non-lossy)"을 주장 — 우리의 "턴을 펼치면 원본 레코드 100% 복원"과 동일한 설계 철학. related work 인용 1순위 후보.
- 긴급 압축 빈도(N턴 세션당 긴급 요약 횟수)는 압축 전략의 평가 지표로 유용 — Claude Code `/compact` 발생 시점을 시각화하는 우리에게는 "측정 대상"이지 "우리 도구의 성능 지표"는 아님.
- OpenHands 컨텍스트 condenser 실측 ([공식 블로그, 2025-04](#)): SWE-bench Verified 부분집합에서 condenser 적용 시 해결률 54% vs 베이스라인 53%(성능 유지)에 API 비용 절반 이하. "성능 손실 없는 비용 50% 절감"의 대표 실측 사례 — 단, 벤더 자체 보고로 명시하고 인용.
- 압축 하의 에이전트 태스크 지표: 압축 연구는 "코드 생성 정확도·파일 수정 정확도" 같은 별도 지표 대신 다운스트림 태스크 성공률(SWE-bench 해결률, DiscoveryBench 등)로 압축 영향을 측정하는 것이 관행. [Evaluating Memory Condensation Strategies for Coding Agents](#)는 OpenHands 기반으로 태스크 성공률 + 톨 호출 성공 + 토큰 비용 + 압축 발생 횟수(compression frequency)를 지표 세트로 사용 — "세션당 압축 횟수"의 학술 출처.
- 톨 호출 신뢰도 경계 ([Control Under Compression](#)): 톨 사용 에이전트의 컨텍스트를 예산별로 압축하면 성공률이 3구간으로 나뉘 — 예산 75%까지는 풀 컨텍스트 대비 1.4%p 이내 유지, 50~35% 구간에서 방법 간 격차가 벌어지며 급락. "어디까지 압축해도 되는가"의 정량 근거.
- 컨텍스트 썩음(context rot) ([Chroma 기술 보고서](#)): 입력이 길어질수록 단순 검색 과제조차 성능이 비선형 하락 — 18개 SOTA 모델에서 확인. 압축이 왜 필요한가의 동기 근거로 인용 (압축 품질 지표 자체는 아님). 일부 2차 자료가 말하는 "Context Rot Index"라는 표준화된 지수는 존재하지 않음 — 현상 명칭일 뿐이니 지표로 인용하지 말 것.
- 유효 컨텍스트 윈도우(모델 윈도우 - 예약 출력 토큰): 실무 개념으로는 유용하나 학술 출처가 약함 — 보고서에 쓴다면 정의만 차용하고 수치 주장은 하지 않기.

3.5 트레이드오프 곡선 보고 관행

단일 수치 대신 곡선/frontier로 보고하는 흐름이 최근 표준으로 자리잡는 중:

- 정확도 vs 압축률 곡선: ACON은 accuracy-peak tokens 평면에서 방법들을 비교. End-to-End Context Compression at Scale은 압축 시간-정확도 평면의 Pareto frontier로 우위를 주장.
- 비용-성능 frontier: The Efficiency Frontier는 컨텍스트 관리 전략 전반을 비용-성능 최적화 문제로 통합 정식화.
- 신뢰도 frontier: 위 Control Under Compression의 3구간 경계 — 압축을 "토큰 절감 문제"가 아니라 "런타임 운영 결정"으로 재정의.
- BranchPort 시사점: 우리가 요약 기능을 평가할 때도 단일 압축률이 아니라 2~3개 압축 수준에서 key-fact recall을 측정해 곡선으로 제시하면 학계 관행과 맞는 보고가 됨 (§4.1 우선순위 5).

4. 권고안 — 결과보고서 지표 구성

보고 틀: 결과보고서의 주장을 둘로 분리하고, 각 주장을 "압축률 + 정보 보존율" 2축 세트로 보고 (학계 관행 차용).

- 주장 A — "흠어진 로그를 한 화면으로" (접기·시각화, 이미 구현): 가독성·탐색 효율.
- 주장 B — "compact로 컨텍스트 비용 절약" (LLM 요약, 기능 도입 가정): 트리에서 선택한 구간을 요약해 새 세션에 이어붙일 압축 컨텍스트를 생성 → 토큰·비용 절감을 품질 보존과 함께 증명.

주장 A의 압축률(레코드→턴 94%)을 주장 B의 근거로 쓰면 안 됨 — 화면 압축이지 컨텍스트 토큰 압축이 아니라서 범주 오류. 두 주장의 수치를 섞지 말 것.

4.1 지표 우선순위

우선순위	지표	주장	측정 방법 · 근거	측정 비용
1	구조적 압축률 (레코드→턴, 실측 ≈94%)	A	즉시 계산 가능 (실측 완료)	즉시
2	재구성 완전성 (무손실 — 오차 0)	A	"턴을 펼치면 원본 100% 복원" — OPENDEV의 'effectively non-lossy'와 동일 철학(§3.4)	즉시
3	토큰 절감률 + 세션당 비용 (USD)	B	원본 구간 vs 압축본 토큰 수(토큰나이저 자동 산출) × API 단가. SelfCompact(30~70%)·OPENDEV(54%)와 동일 형식 — 심사위원에게 가장 직관적	즉시
4	Key-fact recall (수정 파일·명령 보존율)	B	절감률 주장의 필수 짝 — 단독 절감률은 "품질을 버린 것 아니냐"에 무방비. 토큰 레코드에서 정답 기계 추출 → 자동 채점. 우리 데이터 최대 강점	낮음
5	트레이드오프 곡선 (압축 수준 2~3개 × recall)	B	단일 수치보다 설득력 높음. "어디까지 압축해도 안전한가" — Control Under Compression(§3.4)과 비교 가능	낮음
6	SummaC 사실적 일관성	B	recall과 다른 각도(환각 없음)의 품질 보강	낮음~중간
7	과제 완료 시간 A/B (원시 로그 vs BranchPort, Githru식)	A	시간 절약 어필 — 비용 절약(B)과 별도 주장으로 분리	중간 (팀 내부 실험)
8	화면/스크롤 감소율, 질의 보존율	A	원시 JSONL 뷰 vs 턴 뷰 비교	낮음
보조	압축 소요 시간 · 토큰 주체 오귀속 비율 · G-Eval 이진 판정	B	압축 소요 시간은순절감 = 절감 토큰 비용 - 요약 생성 비용 계산에 필요 — 공정 보고용	낮음~중간

4.2 주장 B의 설계 원칙

- baseline 명시: "무압축 대비"는 수치가 크게 나오지만 공정한 비교는 Claude Code 내장 `/compact` 대비. `/compact` 를 정량 평가한 벤치마크는 없으므로(결론 4), 같은 세션에 대해 `/compact` 결과물과 BranchPort compact 결과물의 (토큰 수, key-fact recall)을 나란히 재면 그 자체가 최초 실측 — 대회 차별점.
- 순절감으로 보고: 요약 생성에도 토큰이 들므로, 절감 토큰 비용에서 요약 생성 비용을 뺀 값을 제시해야 공정.
- 최종 구성: 토큰 절감률 + 세션당 비용 + key-fact recall을 2축 세트로, 가능하면 압축 수준별 곡선(우선순위 5)으로.

4.3 조사했으나 제외한 지표

지표	제외 사유
GPU 메모리·KV 캐시·처리량 ·TTFT/TBT-percentile 지연	모델 서빙 지표 — 로컬 시각화 도구가 주장하면 측정 불가능한 수치를 흉내 내는 셈
Perplexity, BLEU/ROUGE(재구성)	소프트 프롬프트 압축 계열 전용 / 우리 구조적 압축은 무손실이라 불필요
SWE-bench류 태스크 성공률	압축 품질의 최종 증거이긴 하나 재현 비용이 대회 일정 대비 과대 — key-fact recall + 소규모 A/B로 갈음
긴급 압축 빈도·세션당 압축 횟수	압축 <i>전략</i> 의 지표. 우리에게 시각화 대상이지 성능 지표가 아님. " /compact 발동 감소" 통제 실험은 여력 있을 때만
Context rot, 유효 컨텍스트 윈도우	지표가 아니라 동기 서사·실무 개념 — 도입부 인용에만 사용, 수치 주장 금지
SummEval 4차원 체크리스트	채점 프레임으로는 유효하나 우선순위 지표들과 중복 — 필요 시 §2 참고

주의사항:

- 벤더 자체 보고 수치(예: AgentPrism "디버깅 시간 80% 감소", OpenHands condenser)를 인용할 땐 "자체 보고 사례"로 명시.
- 일부 원문 PDF(Githru 원문, Aggregate-Driven Trace Visualizations)는 에이전트가 텍스트 추출에 실패해 초록/2차 출처까지만 확인 — 세부 수치 인용 전 직접 열람 필요.
- AI 검색 결과를 조사에 반영할 땐 반드시 원논문으로 역추적할 것. 이번 2차 조사에서 블로그·위키 인용으로 제시된 수치들(SelfCompact 30~70%, PCC F1 63.62/66.29, OPENDEV 54%)은 전부 원논문에서 실재 확인했으나, "FTIT" 같은 비표준 용어가 섞여 있었음(표준: TTFT).

다음 단계

1. 구조적 압축률·재구성 완전성(우선순위 1~2)은 코드에서 바로 산출 가능 — 통계 엔드포인트/화면에 노출 검토.
2. compact 기능 구현 시 토큰 절감률·세션당 비용(우선순위 3)을 함께 출력하도록 설계 — /compact 대비 비교(\$4.2) 까지 가면 최초 실측.
3. Key-fact recall(우선순위 4) 자동 채점 스크립트 설계 — 툴콜 레코드에서 정답 추출 → LLM 요약과 대조.
4. 요약 기능 구현이 진행되면 SummaC 파이프라인 시범 적용.
5. 제출 전 팀 내부 A/B 실험(과제 완료 시간) 1회 실시.

출처 전체 목록

- 요약 품질: BERTScore · SummaC · QAFactEval · QuestEval · G-Eval · SummEval · LLM Judge 신뢰성 2026
- 컨텍스트 압축: LLMlingua-2 · LongLLMLingua · ACON · 정보 보존 · ARC · Prompt Compression in the Wild · SelfCompact (코드) · PCC (ACL 2025) · 경량 압축(엡지 QA) · compaction 커뮤니티 정리

- CLI 에이전트·장문 컨텍스트: [OPENDEV — Building Effective AI Coding Agents for the Terminal](#) · [Chroma Context Rot](#) · [OpenHands Condenser 블로그](#) · [코딩 에이전트 메모리 압축 평가](#) · [Control Under Compression](#) · [LLM 추론 지표 핸드북 \(BentoML\)](#)
- 트레이드오프·frontier: [ACON](#) · [End-to-End Context Compression at Scale](#) · [The Efficiency Frontier](#)
- 대화 요약: [DialSummEval](#) · [대화 요약 오류 분석](#) · [QAGS](#)
- 구조 집약·시각화: [Graph Summarization Survey](#) · [CGS](#) · [Autofolding/TASSAL](#) · [Githru](#) · [PREVis](#) · [Event Sequence 서베이](#)
- 메모리 벤치마크: [LoCoMo/LongMemEval 정리](#) · [mem0 벤치마크 2026](#)